Spec: Fast Corrective + Self-RAG Document Q&A (LangGraph)

Oct 24, 2026 · @Rup

Overview and goals

The system answers a user's question across one or more uploaded documents (typed PDFs, scanned PDFs, or handwritten pages read via OCR) using a LangGraph workflow that combines Corrective RAG (CRAG) with a Self-RAG / LLM-as-a-Judge check. It ships as a self-contained, versioned REST API so the separate backend team can call it without touching its internals, and it must stay fast: a typical answer lands in under about 6 seconds.

Goals

- Answer questions grounded in one or many uploaded documents (typed, scanned or handwritten), with document- and page-level citations.
- Detect weak retrieval and correct it: rewrite the query, and fall back to web search when the document chunks grade too poorly.
- Verify the final answer for grounding (no hallucination) and usefulness (actually answers the question) before returning it.
- Keep latency low by grading all retrieved chunks in one batched LLM call instead of one call per chunk.

Non-goals (v1)

- User accounts, billing or the main product UI (owned by the backend team).
- Multi-turn chat memory (each query is answered independently).
- Non-English handwriting (v2).

Latency targets

Path	Target (p50)	Hard ceiling
Happy path (good chunks, answer passes judge)	3–4 s	6 s
Corrective path (web fallback or one regeneration)	6–8 s	12 s
Ingestion, typed PDF (one-time, 50 pages)	under 15 s	30 s
Ingestion, scanned or handwritten (one-time, per page)	2–4 s	10 s

Inputs and outputs

The API takes one or more documents grouped into a collection, plus a query, and returns one answer with per-document sources and a confidence verdict. All routes are versioned under `/v1` and return JSON.

Endpoints (FastAPI)

Endpoint	Input	Output
`POST /v1/collections`	optional name	`collection_id`
`POST /v1/collections/{collection_id}/documents`	1–10 files (multipart): PDF, PNG, JPG; max 20 MB each	`job_id`, list of `doc_id` + filename (HTTP 202)
`GET /v1/jobs/{job_id}`	none	status per document: `queued`, `ocr`, `indexing`, `ready`, `failed` + page counts
`GET /v1/collections/{collection_id}/documents`	none	documents with status, type (`text`, `scanned`, `handwritten`)
`DELETE /v1/documents/{doc_id}`	none	204

Endpoint	Input	Output
`POST /v1/query`	`collection_id`, `query` (max 1,000 chars), optional `doc_ids` to narrow scope	answer object (below)
`POST /v1/query/stream`	same as `/v1/query`	Server-Sent Events: node progress, answer tokens, final answer object
`GET /v1/health`	none	liveness + dependency checks (LLM, vector store, OCR)

Answer object

```
{
  "answer": "string",
  "sources": [
    {"type": "document", "doc_id": "d_2", "filename": "notes.pdf", "page": 4,
"chunk_id": "c_12", "ocr": true, "ocr_confidence": 0.87},
    {"type": "web", "url": "https://...", "title": "..."}
  ],
  "route": "docs | docs+web | web",
  "verdict": {"grounded": true, "answers_question": true},
  "retries": 0,
  "latency_ms": 3120,
  "request_id": "req_..."
}
```

When the judge still fails after the retry limit, the best answer is returned with `verdict` set to false values and a short caveat line, never an empty response.

Packaging as an API service

The RAG logic lives in a plain Python package with no web code, wrapped by a thin FastAPI layer and shipped as one Docker image, so the backend team integrates through HTTP and the OpenAPI spec alone.

Code layout

```
doc-qa-service/
  core/                 # pure Python, no FastAPI imports
    graph.py            # LangGraph StateGraph + nodes
    ingestion/          # loaders, OCR, chunking, indexing
    ocr/                # OcrProvider interface + implementations
    stores/             # VectorStore, FileStore, JobStore interfaces
    service.py          # answer(query, collection_id, doc_ids) ->
AnswerResult
                        # ingest(files, collection_id) -> JobResult
  api/                  # FastAPI routes, auth, request/response models
  workers/              # background ingestion worker
  config.yaml
  Dockerfile
  docker-compose.yml    # api + worker + redis + vector DB for local runs
```

Integration contract for the backend team

- **OpenAPI:** FastAPI publishes `/openapi.json` and `/docs`; the backend can generate a typed client from it.
- **Versioning:** all routes under `/v1`; breaking changes go to `/v2`, never into `/v1`.
- **Auth:** service-to-service API key in the `X-API-Key` header; the backend handles end-user auth and passes an optional `X-User-Id` for logging only.
- **Stateless API:** no state in process memory; files, indexes and job status live behind store interfaces (local disk + FAISS for dev, S3 + Qdrant/pgvector + Redis for prod) chosen by config.
- **Async ingestion:** uploads return 202 + `job_id` immediately; OCR and indexing run in a worker (Celery/RQ or FastAPI background tasks for v1). Optional `callback_url` gets a POST when the job finishes.
- **Errors:** one JSON shape for every failure: `{"error": {"code": "DOC_NOT_READY", "message": "...", "request_id": "..."}}`, with codes such as `DOC_NOT_READY`, `UNSUPPORTED_FILE`, `OCR_FAILED`, `TIMEOUT`.
- **Config by environment:** model names, API keys, store URLs and limits come from env vars overriding `config.yaml`; no code change to deploy elsewhere.
- **Observability:** every response carries `request_id`; per-node timings go to LangSmith and structured logs.

Because `core/service.py` has no web dependencies, the same functions can also be imported directly as a library if the backend team prefers in-process calls over HTTP.

Architecture

The system has two parts: a one-time ingestion pipeline per uploaded file, and the LangGraph query graph below that runs per question.

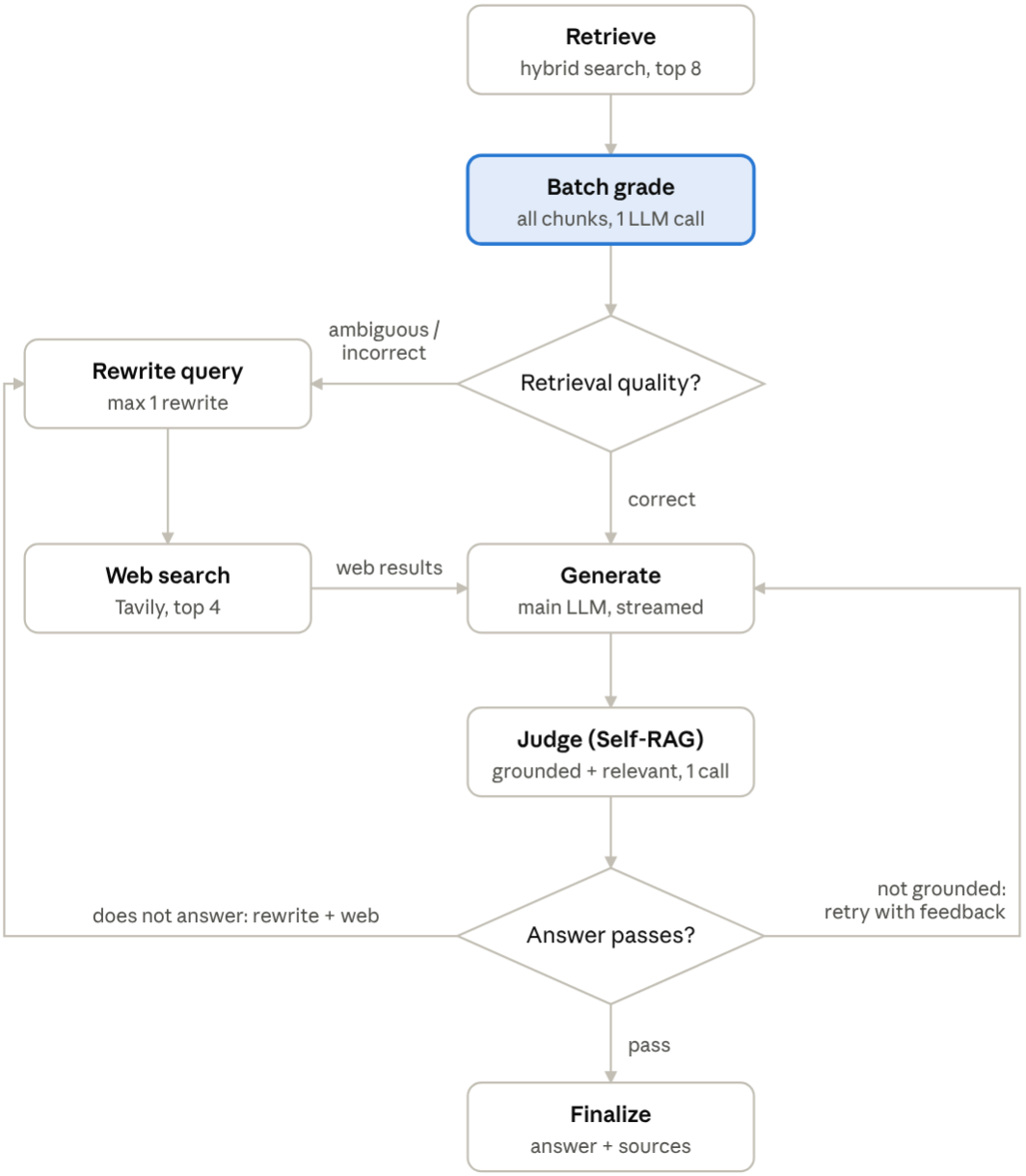
Ingestion (once per uploaded file, in the background worker): each page is routed by type, so typed pages stay fast and only scanned or handwritten pages pay the OCR cost.

1. **Detect:** for each page, check the PDF text layer with PyMuPDF. More than about 50 characters of real text → `text` page; otherwise (or an image upload) → OCR page.
2. **Pre-process OCR pages:** render at 300 DPI, deskew, denoise, boost contrast (OpenCV).
3. **Classify printed vs handwritten:** a cheap first pass with the printed-text OCR engine; low mean confidence (below 0.6) marks the page `handwritten`.
4. **OCR:** printed scans → PaddleOCR (local, fast). Handwritten pages → a vision LLM (e.g. GPT-4o-mini / Gemini Flash) prompted to transcribe exactly, or Azure Document Intelligence Read; chosen via the `OcrProvider` config. Pages run in parallel, up to 8 at a time.
5. **Chunk + index:** chunk the text (keeping `doc_id`, filename, page, `ocr`, `ocr_confidence` in metadata), embed into the collection's vector index, update the BM25 index, cache by file hash.

Pages with OCR confidence below 0.4 are still indexed but flagged, and the generator is told to treat them cautiously.

Query graph (per question):

Weak chunks trigger web search; every answer is judged before it returns



query graph · 8 nodes, 2 decisions, 2 bounded loops

Good chunks go straight to generation; weak ones trigger a query rewrite and web search. The judge either passes the answer or sends it back once; when limits are hit, the best answer is returned with a caveat.

Graph state schema

All nodes read and write one typed state object; counters in it enforce the loop limits.

```
from typing import TypedDict, Literal
from langchain_core.documents import Document

class GraphState(TypedDict):
    collection_id: str
    doc_ids: list[str]           # documents in scope (all in collection if
not given)
    question: str                # original user query (never overwritten)
    search_query: str            # current query used for retrieval / web
search
    chunks: list[Document]       # retrieved chunks; metadata has doc_id,
filename, page, ocr
    grades: list[dict]           # [{chunk_id, score}] from batch grader
    relevant_chunks: list[Document]
    web_results: list[Document]
    retrieval_quality: Literal["correct", "ambiguous", "incorrect"]
    answer: str
    judge: dict                  # {grounded: bool, answers_question: bool,
reason: str}
    rewrite_count: int           # max 1
    generation_count: int        # max 2
    route: str                   # "docs" | "docs+web" | "web"
```

Node specs

Eight nodes make up the query graph; only `batch_grade`, `generate` and `judge` call the main LLMs on the happy path.

Node	Reads	Writes	What it does	Model
`retrieve`	`search_query`, `doc_id`s	`chunks`	Hybrid search (vector + BM25) over all in-scope documents with a doc_id filter, top k = 10, max 4 chunks per document so one file cannot crowd out the rest, then MMR to drop near-duplicates	Embeddings only

Node	Reads	Writes	What it does	Model
`batch_grade`	`question`, `chunks`	`grades`, `relevant_chunks`, `retrieval_quality`	Grades all chunks in ONE call (see Batch grading design)	Small fast LLM
`rewrite_query`	`question`, `grades`	`search_query`, `rewrite_count`	Rewrites the query for retrieval or for web search; one rewrite max	Small fast LLM
`web_search`	`search_query`	`web_results`	Calls the search API, top 3–5 results, keeps snippet + page text trimmed to 1,500 chars each	Tavily API
`knowledge_refine`	`relevant_chunks`, `web_results`	`relevant_chunks`	Optional CRAG strip-level refinement; off by default for speed	Small fast LLM
`generate`	`question`, `relevant_chunks`, `web_results`, `judge`	`answer`, `generation_count`	Writes the answer with inline citations; on a retry, receives the judge's reason as feedback	Main LLM
`judge`	`question`, `answer`, context	`judge`	Self-RAG reflection: grounded? answers the question? One structured call returns both	Small fast LLM
`finalize`	everything	response	Builds the answer object, adds a caveat if the judge failed after retries	None

Judge output (single structured call)

```
{"grounded": true, "answers_question": true, "reason": "All claims trace to
chunks c_3 and c_7."}
```

Combining the hallucination check and the answer-relevance check in one call saves a full LLM round-trip compared with classic Self-RAG, which runs them as two graders.

Batch grading design

All retrieved chunks go to the grader in a single LLM call that returns a score per chunk, turning 8 sequential calls (about 8 × 0.6 s) into one call of roughly 0.8–1.2 s.

How it works

1. Number each chunk with a short ID (`c_0` … `c_7`) and trim each to at most 400 tokens.
2. Send the question plus all numbered chunks in one prompt.
3. Force structured output (Pydantic schema / JSON mode) so the response parses without retries.
4. Score each chunk 0–2 rather than yes/no, so CRAG can tell "correct" from "ambiguous".

Output schema

```
class ChunkGrade(BaseModel):
    chunk_id: str
    score: Literal[0, 1, 2]   # 0 = irrelevant, 1 = partial, 2 = directly
answers

class BatchGrades(BaseModel):
    grades: list[ChunkGrade]
```

Prompt (system)

```
You grade document chunks for relevance to a question.
For EACH chunk id, return a score:
2 = contains information that directly helps answer the question
1 = related but only partially useful
0 = not relevant
Judge each chunk independently. Return every chunk id exactly once.
```

Guardrails

- Batch size: up to 10 chunks per call. If retrieval returns more, split into parallel batches with `asyncio.gather` , not sequential calls.
- Missing or duplicate IDs in the response: treat missing as score 0; never re-call the grader.

- Position bias: shuffle chunk order before grading, then map scores back by ID.
- Grader temperature 0.

Routing logic and loop limits

Routing after grading follows the three CRAG cases, decided purely from the batch scores so no extra LLM call is needed.

Retrieval quality	Rule (from batch scores)	Next step	`route`
Correct	at least 1 chunk scores 2	`generate` with chunks scoring 1–2	`docs`
Ambiguous	no 2s, at least 2 chunks score 1	`rewrite_query` (for web) → `web_search` → `generate` with both document chunks and web results	`docs+web`
Incorrect	all others (mostly 0s)	`rewrite_query` → `web_search` → `generate` with web results only	`web`

After the judge

Judge result	Action
grounded and answers question	`finalize`
not grounded, `generation_count` < 2	`generate` again with the judge's reason as feedback
grounded but does not answer, `rewrite_count` < 1	`rewrite_query` → `web_search` → `generate`
any failure after limits reached	`finalize` with caveat

Hard limits (all configurable)

- Max 1 query rewrite, max 2 generations, so the worst case is about 7 LLM calls.
- Whole-graph timeout of 12 s: on timeout, return the best answer so far.
- LangGraph `recursion_limit` = 12 as a final safety net.

Latency budget and speed optimizations

The happy path spends about 3.5 s across four steps; every optimization below targets either the number of LLM calls or the time the user waits before seeing text.

Step (happy path)	Budget
`retrieve` (embed query + hybrid search)	150 ms
`batch_grade` (1 call, small model)	900 ms
`generate` (main model, streamed)	1,800 ms
`judge` (1 call, small model)	700 ms
Total	**about 3.5 s**

Optimizations, in order of impact

1. **Batch grading:** one grader call for all chunks instead of one per chunk (the core requirement).
2. **Small model for control steps:** grader, rewriter and judge use a fast model; only `generate` uses the larger one.
3. **Merged judge:** grounding and answer-relevance checked in one structured call.
4. **Speculative web search (optional flag):** run `web_search` on the original query in parallel with `batch_grade` via LangGraph fan-out, and discard the results if grading says "correct". Trades one cheap search call for about 1 s saved on the fallback path.
5. **Stream the answer:** stream `generate` tokens to the client; show a "verifying" badge until the judge returns, and replace the answer only if the judge fails.
6. **Ingestion done once:** chunking and embeddings are computed at upload and cached by PDF hash; re-uploads skip ingestion.
7. **Answer cache:** cache `(doc_id, normalized question)` → answer object for repeated questions.
8. **Tight context:** pass only chunks scoring 1–2 to `generate`, trimmed, which shortens prompts and generation time.

Tech stack, config and delivery

The stack is Python with LangGraph for orchestration and FastAPI for serving; every threshold above lives in one config file.

Tech stack

Layer	Choice	Notes
Orchestration	LangGraph (`StateGraph`, conditional edges)	Async nodes throughout
API	FastAPI + Pydantic v2 + SSE streaming	OpenAPI spec auto-generated
Background jobs	Celery or RQ + Redis (FastAPI BackgroundTasks for v1)	Runs ingestion and OCR
PDF parsing	PyMuPDF	Text layer detection, page rendering, page numbers
OCR, printed scans	PaddleOCR	Local, fast; confidence per line
OCR, handwriting	Vision LLM or Azure Document Intelligence Read	Behind an `OcrProvider` interface, swappable by config
Image pre-processing	OpenCV	Deskew, denoise, contrast
Chunking	RecursiveCharacterTextSplitter, 800 chars, 120 overlap	
Vector store	FAISS (dev) / Qdrant or pgvector (prod), one collection per `collection_id`	`doc_id` stored as filterable metadata
Keyword search	BM25 (rank_bm25) merged via reciprocal rank fusion	
Web search	Tavily	Fallback only
LLMs	Small fast model for grade / rewrite / judge; stronger model for generate	Swappable via config
Packaging	Docker image + docker-compose	API and worker from one image
Tracing	LangSmith	Per-node latency

Config (`config.yaml`)

```
api:
  version: v1
  max_files_per_upload: 10
  max_file_mb: 20
```

```
retrieval:
  top_k: 10
  max_chunks_per_doc: 4
  chunk_size: 800
  chunk_overlap: 120
ocr:
  text_layer_min_chars: 50
  dpi: 300
  printed_provider: paddleocr
  handwriting_provider: vision_llm   # or azure_read
  handwriting_conf_threshold: 0.6
  low_conf_flag: 0.4
  max_parallel_pages: 8
grading:
  batch_size: 10
  correct_min_score2: 1
  ambiguous_min_score1: 2
loops:
  max_rewrites: 1
  max_generations: 2
  graph_timeout_s: 12
web_search:
  max_results: 4
  speculative: false
```

Error handling

- LLM call fails or times out (5 s per call): retry once, then skip that step (e.g. no judge → return answer with `verdict` unknown).
- Web search fails: answer from document chunks with a caveat that the documents may not cover the question.
- Malformed grader JSON: treat all chunks as score 1 (ambiguous path).
- OCR fails on a page: mark the page failed, index the rest, report it in the job status; the document still becomes `ready`.
- Query on a collection with no `ready` documents: return `DOC_NOT_READY` (HTTP 409) instead of waiting.

Evaluation

- Test set: 60 question–answer pairs over 5 sample files (3 typed PDFs, 1 scanned PDF, 1 set of handwritten notes), including 10 questions the documents cannot answer (should route to web) and 10 that need two documents together.
- Track: answer correctness, faithfulness (RAGAS), route accuracy, p50 / p95 latency, LLM calls per query.

- OCR quality: character error rate on 20 hand-transcribed handwritten pages, per provider.
- Compare against a baseline that grades one chunk at a time, to show the batch-grading speedup.

Milestones

- [ ] M1: `core` package: ingestion for typed PDFs, multi-document collections, basic retrieve → generate
- [ ] M2: Batch grader + CRAG routing + web fallback
- [ ] M3: Merged Self-RAG judge + retry loop + limits
- [ ] M4: OCR pipeline (printed + handwritten) with background jobs
- [ ] M5: FastAPI layer, `/v1` contract, auth, error shape, Docker image; hand OpenAPI spec to backend team
- [ ] M6: Streaming, caching, speculative search flag, evaluation and latency tuning